University of Information Technology – VNU-HCM
Faculty of Computer Engineering

Computer System

LAB 01

Getting to Know MARS: Registers, Arithmetic Instructions and System Calls

Full name	Phùng Gia Vinh	**Student ID**	26560070
Class	ATBC26	**Practice date**	28/10/2026
Self-assessed score		**GitHub link**	

1. LAB OVERVIEW

Purpose. In this session you will meet the machine that sits underneath every program you have ever used. Starting from an ordinary arithmetic statement such as `a = b + c`, you will see how a processor keeps numbers in registers, how one assembly instruction works on three registers, how that instruction is stored as a single 32-bit number, and how a running program talks to the keyboard and the screen through system calls. Every new idea is explained first and then practised.

> **Situation: the arithmetic inside a smart electricity meter**
> A household electricity meter keeps a running total of the energy used, measured in kilowatt-hours (kWh). The electricity company notes the meter reading at the start of each month. The energy used in a month is the difference between two consecutive readings, and the energy used over several months is the sum of the monthly differences. The small processor inside a smart meter carries out exactly these subtractions and additions on numbers held in registers, and then reports the results. All readings used in this LAB are fictitious.

After this session, you can:

- open, assemble, run, step through and reset a program in MARS, and find the Registers, Text Segment, Data Segment and Run I/O panes;
- name the main MIPS registers (`$zero`, `$t0`–`$t9`, `$s0`–`$s7`, `$v0`, `$a0`, `pc`) and read their contents in decimal and hexadecimal;
- translate simple arithmetic statements such as `a = b + c` into MIPS instructions that use `add`, `sub` and `addi`, with registers playing the role of variables;
- describe how an assembly instruction is stored as a 32-bit machine-code word and read the register numbers inside it;
- use system calls to print text and integers and to read an integer from the keyboard;
- predict, observe and explain the behaviour of a short assembly program.

Table 1. Structure of the session

Part	Content	Suggested time	Score
0	Getting started with MARS	20 min	–
1	Experiment 1 – Meet the registers	30 min	20
2	Experiment 2 – From `a = b + c` to `$t0 = $t1 + $t2`	45 min	30

Part	Content	Suggested time	Score
3	Experiment 3 – Input, output and system calls	50 min	35
4	Wrap-up and submission	15 min	15
–	Debugging, checking and review	20 min	–
Total		**180 min**	**100**

Working conventions

• A yellow cell is where you write an answer or a value. A dashed frame is where you paste a screenshot.

• Program listings appear in grey frames in a monospaced font. In a listing, everything after `#` is a comment. A line that starts with `# >>>` marks a place where you write code.

• Questions are labelled with the type of thinking they ask for: *Understand*, *Apply*, *Analyze* or *Evaluate*.

• Many activities follow the cycle PREDICT → OBSERVE → EXPLAIN. First write down what you expect (PREDICT), then run the program and record what really happens (OBSERVE), and finally explain any difference (EXPLAIN). A wrong prediction is never penalised; the explanation is what earns the marks.

• Tick the box ☐ in a checklist when you have completed the item.

PART 0. GETTING STARTED WITH MARS (20 minutes)

MARS (MIPS Assembler and Runtime Simulator, version 4.5) is a Java program that imitates a MIPS processor. It lets you run assembly programs and watch registers and memory change one instruction at a time, without any real MIPS hardware.

Table 2. Skills checklist

☐	Skill	Hint
☐	Start MARS and create a new file	Double-click `Mars4_5.jar` (or run `java -jar Mars4_5.jar`), then File → New.
☐	Check the settings used in this LAB	In the Settings menu: *Permit extended (pseudo) instructions and formats* ticked; *Initialize Program Counter to global 'main' if defined* ticked; *Delayed branching* unticked; *Popup dialog for input syscalls* unticked.
☐	Assemble a program	Run → Assemble (or the toolbar icon). MARS switches from the Edit tab to the Execute tab.
☐	Run the whole program	Run → Go. Text printed by the program appears in the Run I/O pane at the bottom.
☐	Execute one instruction at a time	Run → Step runs the next instruction; Run → Backstep undoes one; Run → Reset returns to the start. Shortcut keys are shown next to each command.
☐	Identify the panes of the Execute tab	Registers (right), Text Segment (upper left), Data Segment (below it), Messages and Run I/O (bottom).
☐	Switch values between decimal and hexadecimal	Settings → *Values displayed in hexadecimal*.
☐	Save your work	File → Save as… and use the file-name rule given below.

Note

MARS has two views of a program: the Edit tab, where you type, and the Execute tab, where you run it. After every change in the editor you must assemble again. Assembling also resets all registers and memory to their initial state.

Step 0 – Quick check

Type the program in Listing 0 into a new file (use the exact text), save it, assemble it and run it. You will learn what each line means in Experiment 3; for now, only confirm that your installation works.

Listing 0

```
# Listing 0 - quick check
.data
msg:   .asciiz "Hello, MARS!\n"

.text
.globl main
main:
    li   $v0, 4
    la   $a0, msg
    syscall
    li   $v0, 10
    syscall
```

Text that appears in the Run I/O pane:

```
Hello, MARS!
```

File-name rule

Every program you submit is saved as `Lab01_<StudentID>_<Part>.asm`, for example `Lab01_22520001_TN3.asm`. Use `TN2` or `TN3` for the part. Start every file with a comment block containing your name, student ID and a one-line purpose.

EXPERIMENT 1. MEET THE REGISTERS (30 minutes – 20 points)

Situation

The processor in a smart meter does not calculate on numbers stored far away in memory. It first brings the numbers into a small set of very fast storage cells called registers and calculates there. Before programming the meter, you need to know the registers.

Background: registers and the program counter

A processor executes instructions one after another. MIPS has 32 registers, each 32 bits wide (one *word*). Every register has a number from 0 to 31 and a name that starts with `$`. Table 3 lists the registers used in this LAB.

The processor also has a special register, the program counter (`pc`), which holds the address of the next instruction to execute. MARS shows register contents as hexadecimal numbers such as `0x0000002A`: each hexadecimal digit stands for 4 bits, so 8 digits make 32 bits.

Table 3. Registers used in this LAB

Name	Number	Conventional use
`$zero`	0	The constant 0. It always holds 0.

Name	Number	Conventional use
`$v0–$v1`	2–3	Results; `$v0` also carries the service number of a system call (Experiment 3).
`$a0–$a3`	4–7	Arguments; `$a0` also carries the value given to a system call (Experiment 3).
`$t0–$t9`	8–15, 24–25	Temporary values.
`$s0–$s7`	16–23	Variables whose values are kept for a longer time.
`$sp`	29	Stack pointer (used in a later LAB).
`$ra`	31	Return address (used in a later LAB).

Step 1.1 – Find the registers (8 min)

Assemble Listing 0 but do not run it. Look at the Registers pane and complete Table 4 with the register number and its initial value (hexadecimal).

Table 4. Initial register contents

Register	Number	Initial value (hex)
`$zero`	0	0x00000000
`$v0`	2	0x00000000
`$a0`	4	0x00000000
`$t0`	8	0x00000000
`$t9`	25	0x00000000
`$s0`	16	0x00000000
`$sp`	29	0x7fffeffc
`pc`	–	0x00400000

Question 1.1 (Understand) Compare the initial value of `pc` with the *Address* column of the first row of the Text Segment. What do you notice, and what does this tell you about the job of the program counter?

The initial value of pc (0x00400000) exactly matches the Address column of the first instruction in the Text Segment (main). This indicates that the Program Counter (pc) holds the memory address of the next instruction to be fetched and executed by the processor.

Step 1.2 – Predict, observe, explain (15 min)

Open a new file, type Listing 1, save it and assemble it. Before you run anything, fill the PREDICT column of Table 5 with the decimal value that the destination register should hold after each instruction.

Listing 1

```
# Listing 1 - first steps with registers
.text
.globl main
main:
    addi $t0, $zero, 12      # $t0 = $zero + 12
    addi $t1, $zero, 30      # $t1 = $zero + 30
    add  $t2, $t0, $t1       # $t2 = $t0 + $t1
    addi $zero, $zero, 5     # try to put $zero + 5 into $zero
    add  $t3, $zero, $t2     # $t3 = $zero + $t2
    li   $v0, 10             # end of program (see Experiment 3)
    syscall
```

Now use Run → Step to execute the first five instructions one at a time. After each Step, MARS highlights the register that has just changed. Record the observed value of the destination register and the value of `pc` after the instruction. Use Backstep at least once to see an instruction being undone.

Table 5. Predict and observe (decimal values for registers, hexadecimal for pc)

Instruction	Destination register	PREDICT value	OBSERVE value	OBSERVE pc after the instruction
`addi $t0, $zero, 12`	`$t0`	12	12	0x00400004
`addi $t1, $zero, 30`	`$t1`	30	30	0x00400008
`add  $t2, $t0, $t1`	`$t2`	42	42	0x0040000c
`addi $zero, $zero, 5`	`$zero`	5	0	0x00400010
`add  $t3, $zero, $t2`	`$t3`	42	42	0x00400014

Question 1.2 (Apply) Switch the display to hexadecimal. `$t2` now shows `0x0000002a`. Convert this number to decimal by hand and show your calculation.

$0x0000002a = (2 \times 16^1) + (10 \times 16^0) = 32 + 10 = 42_{10}$

Question 1.3 (Analyze) One of your predictions did not match what MARS showed. Which instruction was it? Give your explanation of what happened and what this says about `$zero`.

The instruction addi $zero, $zero, 5 failed to match the prediction (the register value stayed 0). In MIPS architecture, register $zero (Register 0) is hardwired to logic 0. It is read-only in terms of its value, and any write operation to $zero is discarded by hardware.

Question 1.4 (Analyze) By how many does `pc` increase after each instruction? Every MIPS instruction occupies one 32-bit word in memory. Use this fact to explain the number you observed.

The pc increases by **4** (in decimal, or 0x4 in hex) after each instruction. In MIPS, every instruction is exactly 1 word (32 bits = 4 bytes) in length. Because memory is byte-addressed, moving to the subsequent sequential instruction requires advancing the address by 4 bytes.

Screenshot of the Registers pane after the fifth instruction:

Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x00000000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x00000000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x0000000c
$t1	9	0x0000001e
$t2	10	0x0000002a
$t3	11	0x0000002a
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x0040001c
hi		0x00000000
lo		0x00000000

EXPERIMENT 2. FROM a = b + c TO $t0 = $t1 + $t2 (45 minutes – 30 points)

Situation

The firmware of the meter must compute the energy used between two readings. In a high-level language the programmer writes `usage = reading2 - reading1`. The processor, however, executes only instructions that name exactly three registers. This experiment shows how a statement is translated step by step, down to the number that the machine really executes.

Background: from a statement to machine code

The same operation can be written at three levels (Table 6). The program that translates assembly into machine code is called the assembler; MARS contains one.

Table 6. Three levels of the same operation

Level	Example	Written by
High-level statement	`a = b + c`	the programmer
Assembly instruction	`add $t0, $t1, $t2`	the programmer (or a compiler)
Machine code	one 32-bit number, shown as 8 hexadecimal digits	the assembler

Variables become registers

A processor has no named variables. We decide which register stands for which variable, for example `a` → `$t0`, `b` → `$t1`, `c` → `$t2`. The instruction `add` has the form

```
add  destination, source1, source2
```

so `a = b + c` becomes `add $t0, $t1, $t2`. One instruction performs one operation on three registers; a longer expression needs several instructions. For example, `x = p + q - r` with `x`→`$t0`, `p`→`$t1`, `q`→`$t2`, `r`→`$t3` needs two instructions:

```
add  $t0, $t1, $t2      # x = p + q
sub  $t0, $t0, $t3      # x = x - r
```

To put a constant into a register, add it to `$zero`: `addi $t1, $zero, 7` sets `$t1` to 7. To copy a register, add `$zero` to it: `add $t2, $t1, $zero` copies `$t1` into `$t2`.

Step 2.1 – Your first addition (8 min)

Type Listing 2, save it and assemble it. PREDICT the final value of `$t0` before you run it.

Listing 2

```
# Listing 2 - a = b + c
.text
.globl main
main:
    addi $t1, $zero, 7       # b = 7
    addi $t2, $zero, 5       # c = 5
    add  $t0, $t1, $t2       # a = b + c
    li   $v0, 10
    syscall
```

PREDICT – final value of `$t0` (decimal):

12

OBSERVE – execute the three instructions with Step and write down the three registers after each one.

Table 7. Registers after each instruction of Listing 2

After executing	$t0	$t1	$t2
`addi $t1, $zero, 7`	0	7	0
`addi $t2, $zero, 5`	0	7	5
`add  $t0, $t1, $t2`	12	7	5

Step 2.2 – Look at the machine code (12 min)

Select the Execute tab and look at the Text Segment. Each row has the columns Address, Code, Basic and Source: *Source* is the text you typed, *Basic* is the real instruction the processor executes, and *Code* is the 32-bit machine word in hexadecimal. Find the row of `add $t0, $t1, $t2` and note its address and code.

> **Background: the R-type instruction format**
> Instructions such as `add` and `sub` use the R format, in which the 32 bits are divided into six fields:
>
> ```
> | op     | rs     | rt     | rd     | shamt  | funct  |
> | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |
> ```
>
> `op` is the operation code; `rs` and `rt` are the numbers of the first and second source registers; `rd` is the number of the destination register; `shamt` is a shift amount (0 for `add`); `funct` selects the exact operation. To read a hexadecimal code, replace every digit by 4 bits: 0=0000, 1=0001, 2=0010, 3=0011, 4=0100, 5=0101, 6=0110, 7=0111, 8=1000, 9=1001, A=1010, B=1011, C=1100, D=1101, E=1110, F=1111.

Address of the row:

0x00400008

Code of the row (hexadecimal):

0x012a4020

Table 8. Fields of the instruction `add $t0, $t1, $t2`

	op	rs	rt	rd	shamt	funct
Binary	000000	01001	01010	01000	00000	100000
Decimal	0	9	10	8	0	32

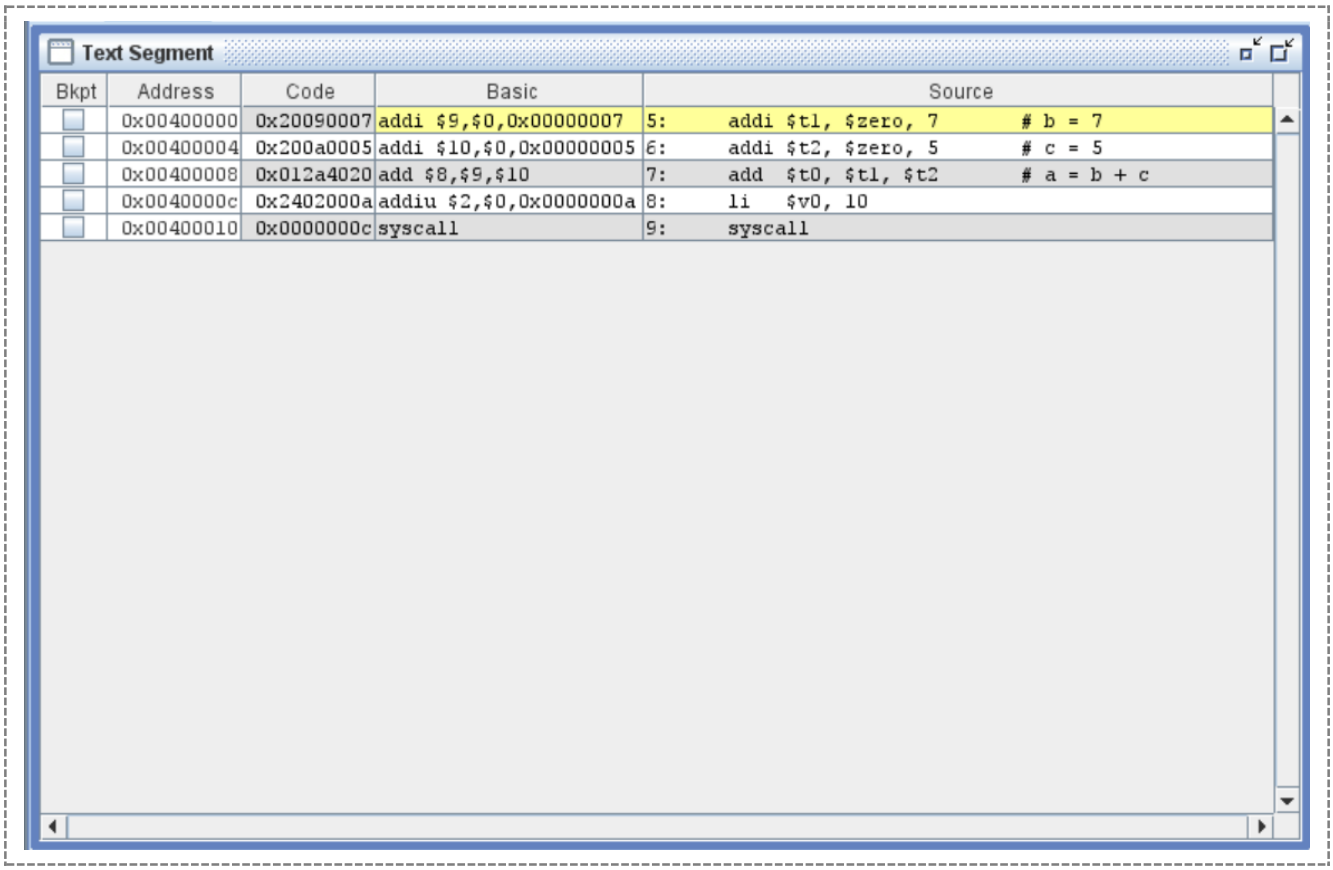
Question 2.1 (Analyze) Compare the decimal values of `rs`, `rt` and `rd` with the register numbers in Table 3. Which operand of the assembly text does each field hold? In which order are the three registers written in the assembly text, and in which order do their fields appear in the machine word?

$rs = 9 corresponds to $t1 (first source operand).
$rt = 10 corresponds to $t2 (second source operand).
$rd = 8 corresponds to $t0 (destination register).
In assembly syntax, the registers appear as:
[operation] [destination], [source1], [source2] (rd, rs, rt).
However, in the machine word layout, they are packed as:
[op] [rs] [rt] [rd] [shamt] [funct]. The hardware expects source registers first, followed by the destination.

Question 2.2 (Understand) Look at the row of `li $v0, 10`. Compare its *Source* and *Basic* columns. Is `li` an instruction that the hardware executes itself? What does the assembler do with it?

li (Load Immediate) is a **pseudo-instruction**; the MIPS hardware does not have a dedicated li opcode. The assembler expands li $v0, 10 into a real basic instruction: ori $v0, $zero, 0x0000000a (or addiu $v0, $zero, 10).

Screenshot of the Text Segment showing the row of `add $t0, $t1, $t2`:



Step 2.3 – Energy used by a household (12 min)

Listing 3 loads four meter readings (in kWh) taken at the start of month 1, the start of month 2, the start of month 3 and the end of month 3. Complete the program so that `$t0`, `$t1` and `$t2` hold the energy used in months 1, 2 and 3, and `$t3` holds the total. Use only `add` and `sub`.

Listing 3

```
# Listing 3 - energy used by a household (fictitious readings, kWh)
# Variable   Register
# reading0   $s0   start of month 1
# reading1   $s1   start of month 2
# reading2   $s2   start of month 3
# reading3   $s3   end of month 3
# usage1     $t0   energy used in month 1
# usage2     $t1   energy used in month 2
# usage3     $t2   energy used in month 3
# total      $t3   usage1 + usage2 + usage3
.text
.globl main
main:
    addi $s0, $zero, 1200
    addi $s1, $zero, 1350
    addi $s2, $zero, 1475
    addi $s3, $zero, 1640

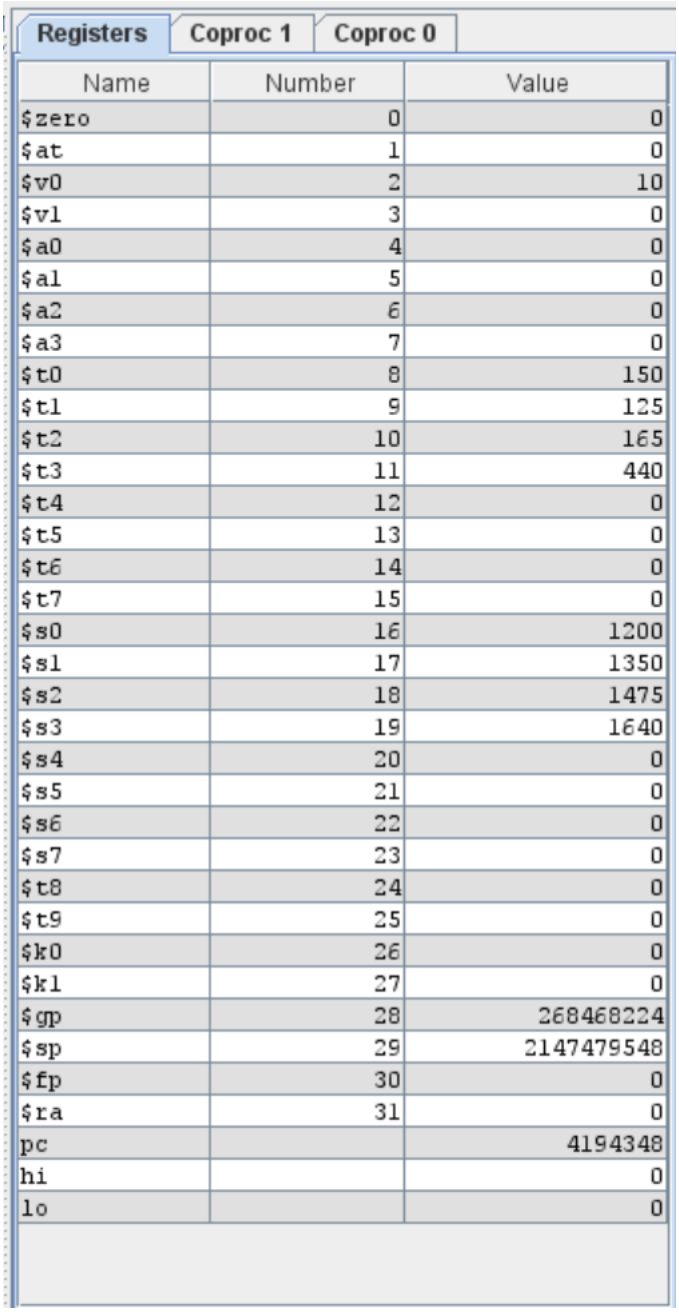
    # >>> usage1 = reading1 - reading0
    # >>> usage2 = reading2 - reading1
    # >>> usage3 = reading3 - reading2
    # >>> total = usage1 + usage2 + usage3   (two add instructions)

    li   $v0, 10
    syscall
```

Check your result

With these readings, after the program has run, `$t0`, `$t1`, `$t2` and `$t3` must contain 150, 125, 165 and 440.

Screenshot of the Registers pane after the program has run:



Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0
$at	1	0
$v0	2	10
$v1	3	0
$a0	4	0
$a1	5	0
$a2	6	0
$a3	7	0
$t0	8	150
$t1	9	125
$t2	10	165
$t3	11	440
$t4	12	0
$t5	13	0
$t6	14	0
$t7	15	0
$s0	16	1200
$s1	17	1350
$s2	18	1475
$s3	19	1640
$s4	20	0
$s5	21	0
$s6	22	0
$s7	23	0
$t8	24	0
$t9	25	0
$k0	26	0
$k1	27	0
$gp	28	268468224
$sp	29	2147479548
$fp	30	0
$ra	31	0
pc		4194348
hi		0
lo		0

Question 2.3 (Analyze) Calculate `reading3 - reading0` by hand and compare it with `total`. Why must the two numbers be equal? Explain in terms of what the meter readings represent.

reading3 − reading0 = 1640 − 1200 = 440

Step 2.4 – Translate statements into instructions (13 min)

Table 9 lists four statements. In each of them the variables `y`, `z` and `w` live in `$s0`, `$s1` and `$s2` and start with the values 8, 5 and 3. Write the MIPS instructions for each statement in the yellow cells, using the destination register shown. Where an intermediate result is needed, use `$t5` as a temporary register. PREDICT the result of each statement by hand before you test it.

Table 9. Statements to translate

Statement	Destination	Your MIPS instructions	PREDICT (decimal)	OBSERVE (decimal)
`x1 = y + z + w`	`$t0`	add $t0, $s0, $s1 add $t0, $t0, $s2	16	16
`x2 = (y + z) - (w + 5)`	`$t1`	add $t1, $s0, $s1 addi $t5, $s2, 5 sub $t1, $t1, $t5	5	5
`x3 = y` (copy)	`$t2`	add $t2, $s0, $zero	8	8
`x4 = z - y`	`$t3`	sub $t3, $s1, $s0	-3	-3

To test your answers, start a new file with the lines below, add your instructions at the marker, run the program and read `$t0`–`$t3`.

Listing 4

```
# Listing 4 - translating statements
.text
.globl main
main:
    addi $s0, $zero, 8       # y = 8
    addi $s1, $zero, 5       # z = 5
    addi $s2, $zero, 3       # w = 3

    # >>> instructions for x1, x2, x3 and x4

    li   $v0, 10
    syscall
```

Question 2.4 (Analyze) The value of `x4` is negative. Show `$t3` in hexadecimal and in decimal. What do the leading hexadecimal digits of a negative number look like, and what does this suggest about how the processor stores negative numbers?

-$t3 in decimal: -3
-$t3 in hexadecimal: 0xfffffffd
-The leading hexadecimal digits are all f (binary 1111). This reflects Two's Complement representation for negative integers: the Most Significant Bit (MSB, bit 31) serves as the sign bit (1 for negative), causing negative numbers with small magnitudes to be sign-extended with leading 1s.

EXPERIMENT 3. INPUT, OUTPUT AND SYSTEM CALLS (50 minutes – 35 points)

Situation

A technician types the readings into the smart meter, and the meter shows the energy used to the customer. A program running on a processor has no screen or keyboard of its own: it must ask the system to read and print on its behalf. This request is called a system call.

Background: program memory and text

A MIPS program has two parts. The `.text` part holds the instructions. The `.data` part holds the data that the program uses, such as messages. In MARS the data part starts at address `0x10010000`.

A label is a name for an address. The directive `.asciiz "text"` stores the characters of a message followed by one extra byte of value 0, which marks the end of the message. The pseudo-instruction `la $a0, label` puts the address of the label into `$a0`.

Background: how to make a system call

1. Put the service number in `$v0`. 2. Put the argument, if any, in `$a0`. 3. Execute `syscall`. 4. If the service returns a result, read it from `$v0`.

In MARS, the simulator plays the role of the operating system and performs the service. Table 10 lists the four services used in this LAB.

Table 10. System-call services used in this LAB

Service	$v0	Argument	Result
Print integer	1	`$a0` = integer to print	–
Print string	4	`$a0` = address of a string that ends with 0	–
Read integer	5	–	the integer typed by the user, in `$v0`
Exit program	10	–	–

Step 3.1 – Print text and a number (12 min)

Type Listing 5, save it and assemble it. PREDICT exactly what will appear in the Run I/O pane.

Listing 5

```
# Listing 5 - print a message and an integer
.data
msg:   .asciiz "Usage (kWh): "
nl:    .asciiz "\n"

.text
.globl main
main:
    addi $t0, $zero, 150
    li   $v0, 4              # service 4: print string
    la   $a0, msg            # argument: address of msg
    syscall
    li   $v0, 1              # service 1: print integer
    add  $a0, $t0, $zero     # argument: the number to print
    syscall
    li   $v0, 4              # print a new line
    la   $a0, nl
    syscall
    li   $v0, 10             # service 10: exit
    syscall
```

PREDICT – text in the Run I/O pane:

Usage (kWh): 150

OBSERVE – run the program. Then open Settings → Show Labels Window (symbol table) and read the addresses of the two labels.

Table 11. Label addresses

Label	Address (hex)
`msg`	0x10010000
`nl`	0x1001000e

Screenshot of the Run I/O pane after the program has run:

Paste screenshot here

Question 3.1 (Understand) Which register told the system *which service* was wanted, and which register carried *what to print*? What would happen to the meaning of `$a0` if you changed the service number?

Register $v0 holds the service code telling MARS which operating system operation to invoke (e.g., 4 for string output, 1 for integer output). Register $a0 supplies the primary argument. Changing $v0 fundamentally alters how $a0 is interpreted: under service 1, the processor reads $a0 directly as a numerical value; under service 4, it interprets $a0 as a pointer/memory address pointing to an ASCII byte sequence.

Question 3.2 (Apply) Subtract the address of `msg` from the address of `nl`. Count the characters of the message `Usage (kWh): ` (including spaces) and compare with your result. How do you explain the difference between the two numbers?

The address of nl is 0x1001000e and msg is 0x10010000. 0x1001000e - 0x10010000 = 0xe (which is 14 in decimal).Comparison & Explanation: The character count for "Usage (kWh): " is exactly 13 characters (including spaces). The subtraction result is 14, making a difference of 1. This difference occurs because the string was defined using the .asciiz directive, which automatically appends a null-terminator character (\0) at the end of the string to tell the processor where the text ends. This null character occupies the 14th byte in memory.

Step 3.2 – Read two numbers and compute the usage (15 min)

Listing 6 reads the previous meter reading from the keyboard and stores it in `$s0`. Complete the program so that it also reads the current reading into `$s1`, computes `usage = current - previous` in `$t0`, and prints the result in the form shown below the listing.

Listing 6

```
# Listing 6 - energy used between two readings
.data
prompt1: .asciiz "Enter previous reading (kWh): "
prompt2: .asciiz "Enter current reading (kWh): "
label:   .asciiz "Usage (kWh): "
newline: .asciiz "\n"

.text
.globl main
main:
    # --- read the previous reading into $s0 ---
    li   $v0, 4
    la   $a0, prompt1
    syscall
    li   $v0, 5              # service 5: read integer, result in $v0
    syscall
    add  $s0, $v0, $zero     # keep the value in $s0

    # >>> read the current reading into $s1 (print prompt2, then service 5)

    # >>> usage = current - previous, into $t0

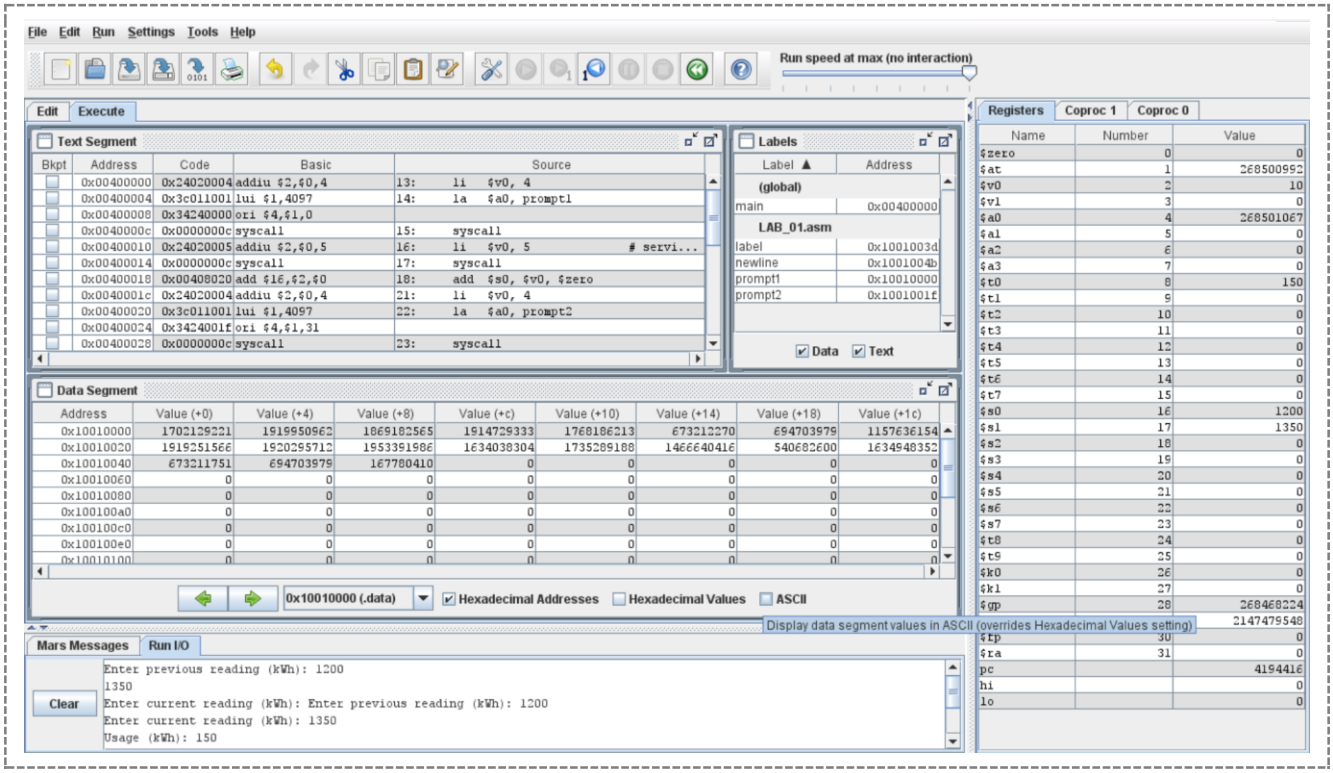
    # >>> print label, then the usage (service 1), then newline

    li   $v0, 10
    syscall
```

Check your result

Run the program and type `1200` and `1350` when asked. The last line in the Run I/O pane must be `Usage (kWh): 150`. Type each number in the Run I/O pane and press Enter.

Screenshot of the Run I/O pane for this test:



Step 3.3 – Three-month summary with your own data (15 min)

Extend your program so that it reads four readings in order (reading0 to reading3) and then prints the usage of each month and the total. Start from the `.data` section below and keep the register plan of Listing 3 (`$s0–$s3` for the readings, `$t0–$t3` for the results). The output must have this form:

```
Month 1: <usage1>
Month 2: <usage2>
Month 3: <usage3>
Total:   <total>
```

Listing 7 (data section)

```
.data
prompt: .asciiz "Enter reading (kWh): "
m1:     .asciiz "Month 1: "
m2:     .asciiz "Month 2: "
m3:     .asciiz "Month 3: "
tot:    .asciiz "Total:   "
nl:     .asciiz "\n"
```

First test the program with the readings 1200, 1350, 1475 and 1640; the four results must be 150, 125, 165 and 440. Then take D, the last digit of your student ID, and look up your own readings in Table 12. PREDICT the four results by hand, run the program with your readings, and record what you observe.

Table 12. Personal readings (D = last digit of the student ID)

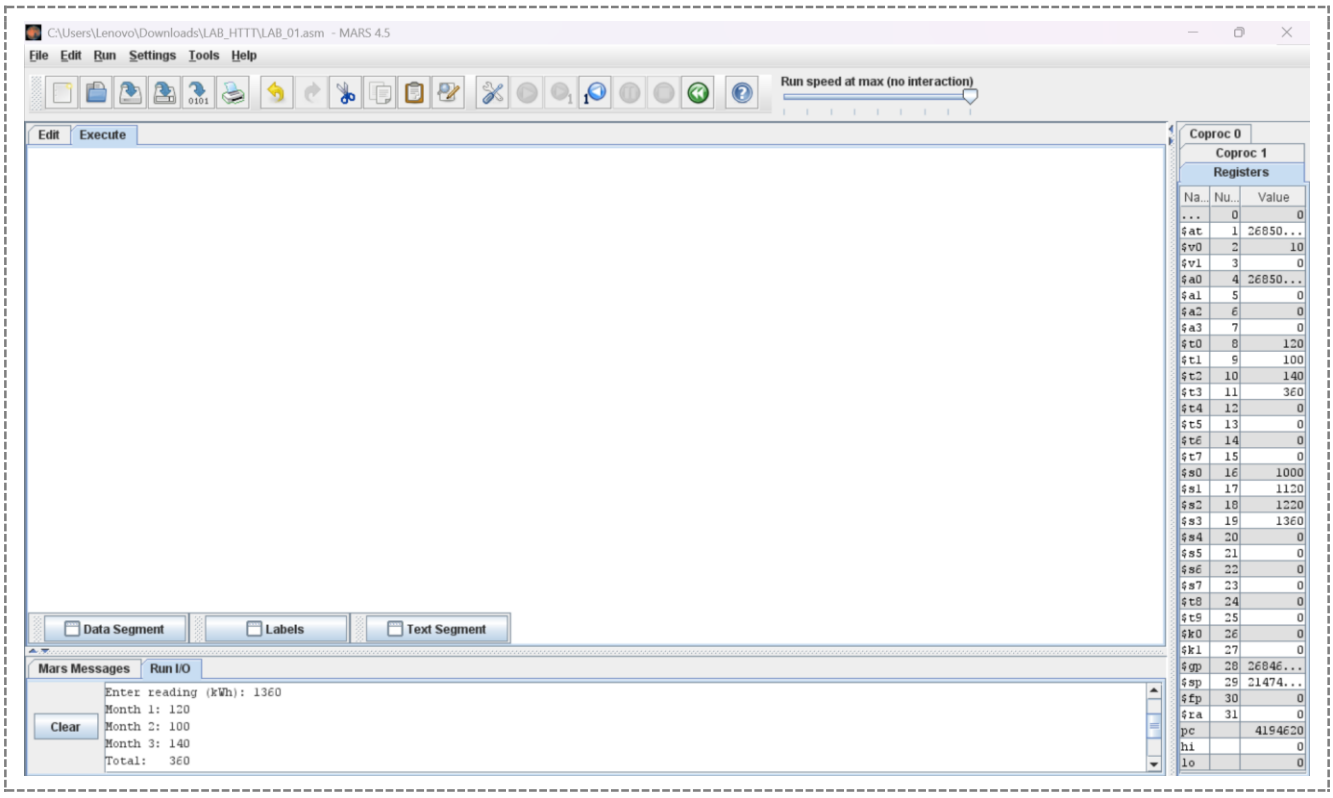
D	reading0	reading1	reading2	reading3
0	1000	1120	1220	1360
1	1020	1143	1248	1390
2	1040	1166	1276	1420
3	1060	1189	1304	1450

D	reading0	reading1	reading2	reading3
4	1080	1212	1332	1480
5	1100	1235	1360	1510
6	1120	1258	1388	1540
7	1140	1281	1416	1570
8	1160	1304	1444	1600
9	1180	1327	1472	1630

Table 13. Results for your own readings

My D	Result	Month 1	Month 2	Month 3	Total
0	**PREDICT**	1120-1000=120	1220-1120=100	1360-1220=140	360
0	**OBSERVE**	120	100	140	360

Screenshot of the Run I/O pane with your own readings:



Step 3.4 – Break it on purpose (8 min)

Programmers learn a great deal from errors. For each experiment in Table 14, PREDICT what will happen, make the change, run the program and OBSERVE the result. Restore the program afterwards.

Table 14. Controlled experiments

Experiment	PREDICT	OBSERVE
(a) Run your Step 3.2 program and enter the readings in the wrong order: `1350` first, then `1200`.	1200 – 1350 = -150	Usage (kWh): -150

Experiment	PREDICT	OBSERVE
(b) In Listing 5, delete the last two instructions (`li $v0, 10` and `syscall`) and run it.	Program won't exit cleanly; executes past program bounds into uninitialized memory until runtime trap.	Error in ...: dropped off bottom that was not a text/instruction segment!
(c) In Listing 5, change the service number before printing the integer from `1` to `4`, and run it.	Program crashes with an illegal memory address error trying to read address 150 as a string pointer.	Error in ...: invalid address [0x00000096] while fetching string

Question 3.3 (Analyze) Explain your observation in experiment (c). What did the system try to treat as an address, and why did the program fail?

In experiment (c), service 4 expects $a0 to be a valid memory pointer to a null-terminated ASCII string. Instead, $a0 contained the raw numeric integer 150 (0x00000096). The simulator tried to read string characters from memory address 0x00000096, which is in the restricted/unmapped low memory, not in the user data segment (0x10010000), causing a segmentation fault/memory address exception.

PART 4. WRAP-UP AND SUBMISSION (15 minutes – 15 points)

Question 4.1 (Understand) In your own words, describe how the statement `a = b + c` becomes something the processor can execute. Use the terms *register*, *assembly instruction*, *assembler* and *machine code*.

A high level statement like a = b + c is translated to CPU registers representing variables (e.g. $t0, $t1, $t2) This operation is defined by the programr or compiler by writing an assembly instruction (add $t0, $t1, $t2). The assembler converts the mnemonic and register symbols into machine code, a 32-bit binary instruction word. The processor then fetches, decodes and executes the instruction using electronic logic gates.

Question 4.2 (Apply) Write the instructions that print the integer held in `$t5` followed by a new line, assuming that a string labelled `nl` exists in the `.data` section.

```
li   $v0, 1
add  $a0, $t5, $zero
syscall

li   $v0, 4
la   $a0, nl
syscall
```

Question 4.3 (Evaluate) Your Step 3.3 program contains several almost identical blocks of instructions. In two or three sentences, suggest a feature that would make such a program shorter, and say what the processor would need to be able to do to support it.

An array/memory buffer (or modular procedures/functions) loop structure would considerably reduce the length of the program by eliminating redundant input/output sections. For this, the processor needs branch/jump instructions (e.g. beq, bne, j or jal/jr) to change the flow of the program, and memory access instructions (lw, sw) to sequentially address stored values.

Table 15. Submission checklist

☐	Item
☐	Files `Lab01_<StudentID>_TN2.asm` and `Lab01_<StudentID>_TN3.asm` are saved with the correct names.
☐	Every file begins with a comment block (name, student ID, purpose) and every program assembles without errors.
☐	Every `# >>>` marker has been replaced by your own code and the register plan is documented in comments.
☐	This document is complete: all yellow cells are filled and every dashed frame contains a screenshot.
☐	The `.asm` files are pushed to your GitHub repository, and the repository link is written in the information table on the first page.
☐	The document and the repository link are submitted in the way announced by your instructor.

APPENDIX. INSTRUCTION REFERENCE

Table 16. Instructions used in this LAB

Instruction	Example	Meaning
`add`	`add $t0, $t1, $t2`	`$t0 = $t1 + $t2`
`sub`	`sub $t0, $t1, $t2`	`$t0 = $t1 - $t2`
`addi`	`addi $t0, $t1, 20`	`$t0 = $t1 + 20`
`syscall`	`syscall`	ask the system to perform the service whose number is in `$v0`
`li` (pseudo)	`li $t0, 7`	`$t0 = 7`; the assembler turns it into a real instruction
`la` (pseudo)	`la $a0, label`	`$a0` = address of `label`; the assembler turns it into real instructions